CSDS 2026 – HW 1

Remark: *Using AI to find a more appropriate method for Question 8 and adjust RMarkdown formatting.*

Load Data

```
df <- read.table("customers.txt", header = TRUE)
head(df)
```

```
##   age
## 1  49
## 2  69
## 3  41
## 4  73
## 5  45
## 6  71
```

1. What is the 5th element in the original list of ages?

```
age <- df$age
fifth_element <- age[5]
fifth_element
```

```
## [1] 45
```

2. What is the fifth lowest age?

I think "fifth lowest age" can be interpreted in two ways, depending on whether we consider duplicates:

Case 1: Under the literal interpretation, keep duplicates and sort ages in ascending order and select the fifth element.

```
sorted_ages <- sort(age)
fifth_lowest_age <- sorted_ages[5]
fifth_lowest_age
```

```
## [1] 19
```

Case 2: But if it reflects true ranking, it should refer to the fifth "distinct" age, which requires removing duplicates before sorting.

```
sorted_unique_ages <- sort(unique(age))
fifth_lowest_distinct_age <- sorted_unique_ages[5]
fifth_lowest_distinct_age
```

```
## [1] 22
```

3. Extract the five lowest ages together

Following Q2, since the question can be interpreted either with or without removing duplicates, the result will be different under these two situations:

Case 1: Sort without removing duplicates first

```
five_lowest_ages <- sorted_ages[1:5]
five_lowest_ages
```

```
## [1] 18 19 19 19 19
```

Case 2: Sort after removing duplicates

```
five_lowest_distinct_ages <- sorted_unique_ages[1:5]
five_lowest_distinct_ages
```

```
## [1] 18 19 20 21 22
```

4. Get the five highest ages by first sorting them in decreasing order first

Following Q2 & Q3, the result of the "five highest" ages will also vary under these two interpretations:

Case 1: Sort in decreasing order without removing duplicates first

```
sorted_ages_desc <- sort(age, decreasing = TRUE)
five_highest_ages <- sorted_ages_desc[1:5]
five_highest_ages
```

```
## [1] 85 83 82 82 81
```

Case 2: Sort in decreasing order after removing duplicates

```
sorted_unique_ages_desc <- sort(unique(age), decreasing = TRUE)
five_highest_distinct_ages <- sorted_unique_ages_desc[1:5]
five_highest_distinct_ages
```

```
## [1] 85 83 82 81 80
```

5. What is the average (mean) age?

```
mean_age <- mean(age, na.rm = TRUE)
mean_age
```

```
## [1] 46.80702
```

6. What is the standard deviation of ages?

```
standard_deviation_age <- sd(age, na.rm = TRUE)
standard_deviation_age
```

```
## [1] 16.3698
```

7. Make a new variable called `age_diff`, with the difference between each age and the mean age

```
age_diff <- age - mean_age
head(age_diff)
```

```
## [1]  2.192982 22.192982 -5.807018 26.192982 -1.807018 24.192982
```

8. What is the average "difference between each age and the mean age"?

If we simply calculate with the raw differences, the average will be zero due to the offset of positive and negative values, which is not meaningful.

```
mad_age_diff <- mean(age_diff, na.rm = TRUE)
mad_age_diff
```

```
## [1] -8.191721e-16
```

Instead, I would like to use *MAD* to measure the average difference, which can better reflect the true deviation magnitude.

MAD (Mean Absolute Deviation)

Reference: https://vocus.cc/article/6767df46fd897800010f0954

$$\text{MAD} = \frac{1}{n}\sum_{i=1}^{n}|age_i - \bar{age}|$$

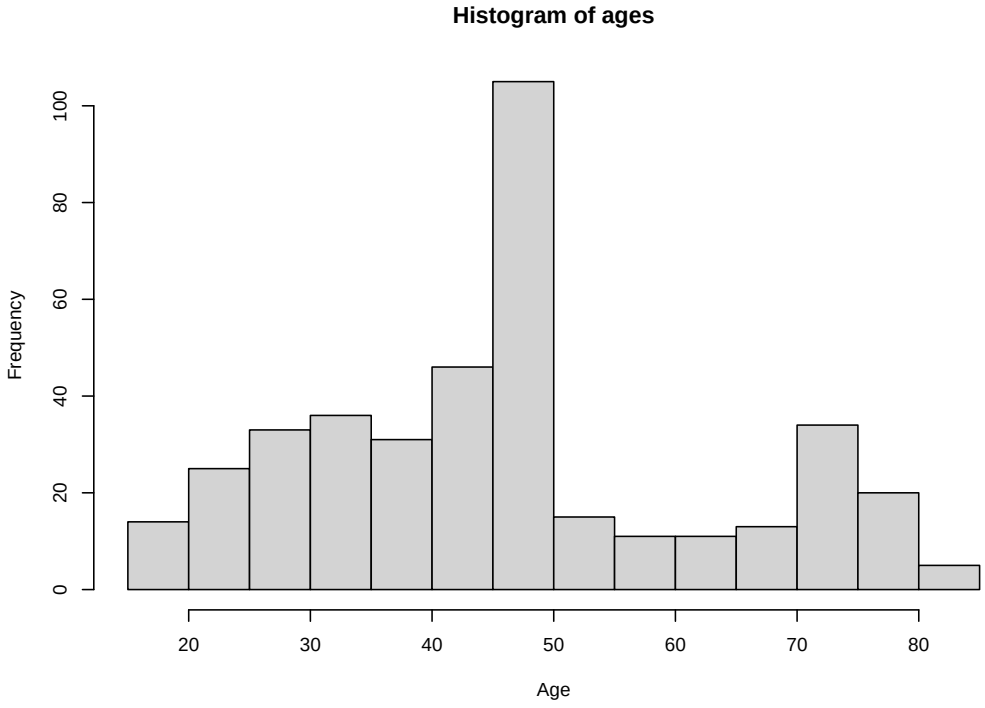
```
mad_age_diff <- mean(abs(age_diff), na.rm = TRUE)
mad_age_diff
```

```
## [1] 12.66948
```

9. Visualize the raw data

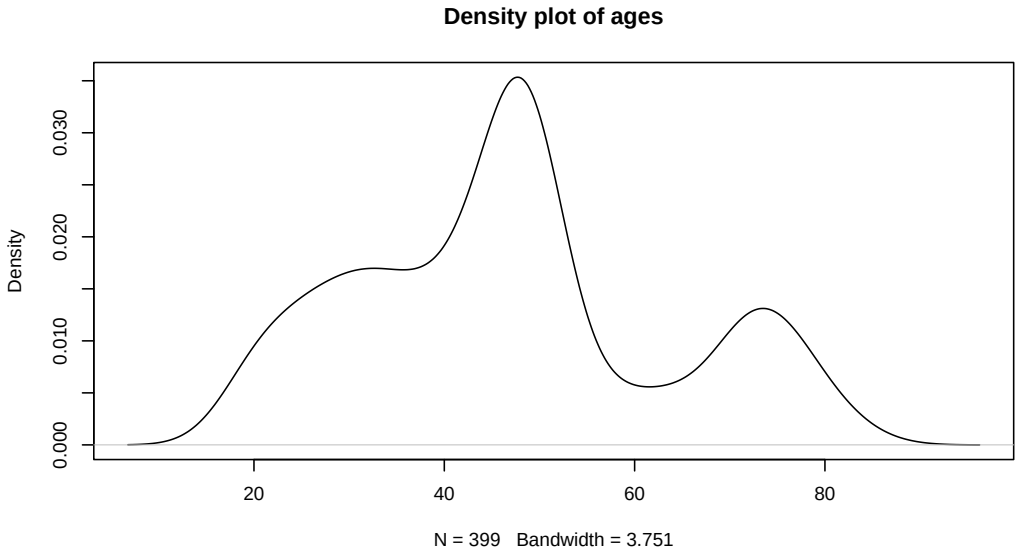
(a) Histogram

```
hist(age, main = "Histogram of ages", xlab = "Age")
```



(b) Density plot

```
plot(density(age), main = "Density plot of ages")
```



(c) Boxplot + stripchart

```
boxplot(age, horizontal = TRUE, main = "Boxplot of ages", xlab = "Age")
stripchart(age, method = "stack", add = TRUE)
```